

Table of Content

Introduction/Objectives	03
System Analysis	04
o Identification of Need	04
o Preliminary Investigation	04
o Feasibility Study	04
o Project Planning	05
Software Requirement Specifications (SRS)	06
o Introduction	06
o Software Engineering Paradigm Applied	07
o Functional Requirements	08
o High Level System Architecture	08
System Design	09
Coding	11
System Security Measures	12
Cost Estimation	12
Appendices	13

Introduction/Objectives

In today's competitive job market, the overwhelming volume of resumes poses a significant challenge for recruiters and employers seeking to identify the most qualified candidates efficiently. **Resume Filter** is a project designed to automate the processing of resumes. It uses Python to extract text from PDF resumes and then uses ChatGPT to identify key information like personal details, education, experience, and skills. The project is built on the Django web framework and offers user-friendly interface to upload and to see the results. It helps the Human Resource (HR) team to streamline the recruitment process by providing a structured Excel sheet of extracted information about each resume and it saves the time and improving candidate assessment efficiency.

System Analysis

Identification of Need

The increasing volume of resumes in the modern recruitment process presents significant challenges for recruiters. Manual screening is time-consuming, inconsistent, and prone to human error, making it difficult to accurately assess all candidates. The need for automation and advanced technology to efficiently and accurately process resumes became apparent, leading to the development of "Resume Filter," a solution aimed at automating the extraction of crucial information from resumes to streamline the recruitment process.

Preliminary Investigation

A preliminary investigation was conducted to analyse the challenges faced by recruiters. It was found that traditional manual screening processes were inefficient, prone to bias, and unable to scale with the volume of resumes. A thorough analysis of available technologies revealed that using a combination of Python for PDF text extraction, ChatGPT for natural language processing (NLP)-based entity extraction, and the Django web framework for developing APIs and a user-friendly interface could provide an effective solution.

Feasibility Study

The feasibility study assessed the project from multiple angles:

- **Technical Feasibility:** The tools chosen—Python, ChatGPT, and Django—are robust, widely supported, and capable of handling the complexities of resume processing. The integration of NLP for entity extraction and automation ensures that the project is technically viable.
- **Operational Feasibility:** The implementation of the "Resume Filter" system will significantly improve the recruitment process by reducing manual workload, improving accuracy, and enhancing decision-making. Recruiters can adopt this system without significant changes to their workflows.
- **Economic Feasibility:** Automating the resume filtering process reduces the need for extensive HR resources dedicated to manual screening. The investment in the technology is justified by the cost savings in terms of time and improved hiring outcomes, leading to long-term financial benefits.

- **Schedule Feasibility:** The project can be developed and implemented within a reasonable timeframe, given the scope of the system and the availability of technology and expertise.

Project Planning

The project was planned to be executed in phases:

1. **Requirement Analysis & Design:** Identify the key entities to be extracted from resumes, such as personal details, qualifications, and work experience. Design the system architecture, incorporating the PDF text extractor, ChatGPT-based NLP, and Django web framework.
2. **Development:** Develop the core functionality for PDF text extraction, implement ChatGPT for entity extraction, and build the Django-based API and web interface for interaction with the system.
3. **Testing:** Test the system with various resumes in different formats to ensure accurate entity extraction and seamless integration of all components.
4. **Deployment:** Deploy the system to a production environment, ensuring scalability to handle large volumes of resumes efficiently.
5. **Maintenance & Improvement:** Continuously monitor system performance and gather user feedback for ongoing improvements and updates.

This structured approach ensures that the project meets its objectives of automating resume filtering and improving the overall efficiency of the recruitment process.

Software Requirement Specifications (SRS)

Introduction

Purpose

The "Resume Filter" project, a tool designed to automate the extraction of key information from resumes in PDF format. The extracted information will be presented in a structured Excel sheet, streamlining the recruitment process and enhancing the efficiency of candidate evaluation.

Scope

The "Resume Filter" project will focus on:

- Extracting textual content from PDF resumes using a Python PDF text extractor package.
- Utilizing ChatGPT for entity extraction to identify key information such as personal details, educational qualifications, work experience, and skills.
- Developing a Django-based web application with APIs and a user-friendly UI for interacting with the system.

Definitions, Acronyms, and Abbreviations

- **NLP:** Natural Language Processing
- **API:** Application Programming Interface
- **UI:** User Interface
- **HR:** Human Resources
- **PDF:** Portable Document Format

References

- Python PDF text extraction libraries
- OpenAI's ChatGPT API documentation
- Django web framework documentation

Overview

This document outlines the system requirements for the "Resume Filter" project. It covers the software engineering paradigms applied, system architecture, functional and non-functional requirements, and external interfaces.

Software Engineering Paradigm Applied

The software engineering paradigm applied to the "Resume Filter" project is the Iterative and Incremental Development model. This paradigm was chosen to accommodate the project's complex and evolving requirements, allowing for continuous refinement of functionalities such as resume parsing, entity extraction, and user interaction. It also supports the integration of various technologies including Python, Django, and ChatGPT.

Justification

1. **Flexibility:** The Iterative and Incremental Development model allows the project to adapt to changing requirements and technologies. For example, different entity extraction techniques or API enhancements can be incorporated as the system evolves.
2. **Risk Management:** By breaking the project into manageable increments, risks related to complex technologies like NLP and AI are mitigated. Any issues that arise can be addressed in subsequent iterations.
3. **Continuous Improvement:** Each iteration provides an opportunity for refining the system, improving the accuracy of entity extraction and enhancing the user interface for better usability.

Application of Paradigm

- **Initial Iteration:** The first iteration focuses on implementing the core functionalities like resume parsing using Python PDF extraction packages and basic entity extraction with ChatGPT.
- **Subsequent Iterations:** Future iterations will incrementally add features such as more advanced entity extraction, additional data formats, improved scalability, user authentication, API endpoints, and an enhanced Web UI built on Django.
- **Feedback Integration:** User feedback will be collected after each iteration and incorporated into the development of subsequent features, ensuring that the system evolves to meet user expectations.

Functional Requirements

Resume Parsing

- The system must extract text from PDF resumes using a Python-based text extraction library.

Entity Extraction

- ChatGPT will extract relevant entities such as personal details, education, work experience, and skills from the parsed resume data.

Data Output

- The extracted entities must be outputted into a structured Excel sheet format, providing recruiters with easy access to candidate information.

User Authentication

- The system will utilize the 'allauth' package to authenticate users accessing the Web UI and API.

API Development

- The Django framework will provide API endpoints for the upload of resumes and retrieval of processed data.

Web Interface

- The system must feature a user-friendly Web UI built on Django that allows recruiters to upload resumes and review the extracted data.

High-Level System Architecture

The system is composed of the following components:

- **Python PDF Extractor:** Responsible for extracting textual content from PDF resumes.
- **ChatGPT Entity Extractor:** Processes extracted text to identify relevant entities.
- **Django API:** Provides an interface for uploading resumes and retrieving structured data.
- **Django Web UI:** User-friendly interface for recruiters to interact with the system.

System Design

Modularization Details:

The "Resume Filter" project is organized into several modules to maintain scalability and ease of maintenance:

1. PDF Text Extraction Module:

- **Description:** This module utilizes a Python package (e.g., PyPDF2, pdfplumber) to extract text from resumes in PDF format.
- **Responsibility:** To clean, preprocess, and structure raw text data for further processing.
- **Inputs/Outputs:** Inputs are PDF files, and outputs are structured text.

2. Entity Extraction Module:

- **Description:** Employs ChatGPT (or similar NLP models) to identify entities like names, contact information, educational qualifications, work experience, and skills.
- **Responsibility:** Ensures accurate extraction of structured information from resumes.
- **Inputs/Outputs:** Inputs are structured text, and outputs are extracted entities in JSON format.

3. Database Module:

- **Description:** Uses Django's ORM to manage database interactions.
- **Responsibility:** Stores the extracted entities and information related to resumes.
- **Inputs/Outputs:** Inputs are JSON objects, and outputs are database entries.

4. Excel Export Module:

- **Description:** Converts extracted entities into structured Excel sheets for use by recruiters.
- **Responsibility:** Formatting and generating Excel sheets using Python packages like pandas or openpyxl.
- **Inputs/Outputs:** Inputs are database objects, and outputs are Excel sheets.

5. API and Web UI Module:

- **Description:** Uses Django to provide REST APIs and a user-friendly interface for interaction with the system.

- **Responsibility:** Handles user requests for uploading resumes, viewing extracted data, and exporting results.
- **Inputs/Outputs:** Inputs are API calls or user input from the UI, and outputs are either JSON responses or rendered HTML pages.

Data Integrity and Constraints:

- **Entity Validation:** Ensuring that the data extracted (e.g., contact info, work experience) is accurate and correctly categorized. This includes validating against predefined patterns like email formats, date ranges, and phone numbers.
- **Database Constraints:**
 - **Uniqueness Constraints:** E.g., no duplicate records for resumes based on candidate email or name.
 - **Foreign Key Constraints:** Relationships between tables such as candidates and resumes are enforced with foreign keys.
 - **Field Constraints:** Ensuring fields such as dates of employment or GPA are within realistic limits.
- **Consistency:** Every time a resume is processed, old records are either updated or new ones are created, ensuring no redundant data is stored.

User Interface Design:

- **Dashboard:** Displays an overview of the system, including processed resumes, status of extractions, and options for exporting data.
- **Resume Upload Page:** Allows users to upload resumes in bulk or individually. This page provides file upload features, drag-and-drop functionality, and validation checks for file types.
- **Data Viewing Pages:**
 - **Candidate Details:** Displays information extracted from the resume such as name, contact details, education, experience, and skills.
 - **Excel Export Interface:** Provides options for filtering candidates based on skills or experience and exporting their data to an Excel sheet.

Database Design:

1. **Candidate Table:**
 - **Fields:** candidate_id (PK), name, email, phone, location
2. **Resume Table:**

- **Fields:** resume_id (PK), candidate_id (FK), resume_file, upload_date

3. Table for extracted Data:

- **Fields:** resume_id (PK), candidate_id (FK), Education, Skills, Summary_of_Resume

Coding

Standardization of the Coding

In the "Resume Filter" project, code standardization is paramount to ensure consistency, readability, and maintainability throughout the entire codebase. Following PEP 8, the official Python style guide, the project adheres to standardized naming conventions, indentation, and code formatting. This facilitates collaborative development, making the code easier to review and extend. Furthermore, the use of docstrings and type annotations across functions and classes guarantees that developers can comprehend the purpose and expected input/output of every module at a glance. In future, I am planning to apply design patterns such as Singleton and Factory where appropriate to ensure the standardization of object-oriented programming (OOP) practices across the application.

Code Efficiency

Efficiency is at the core of the Resume Filter's Python implementation. By employing optimized algorithms for PDF text extraction and entity parsing, we minimize time complexity and ensure swift processing of resumes. The project leverages Python's built-in libraries such as multiprocessing and concurrent.futures to parallelize time-intensive tasks, such as text extraction from multiple PDF files, ensuring faster execution times in large-scale recruitment environments. Additionally, the Django framework is optimized by employing efficient querysets and caching mechanisms to reduce database queries and boost performance in the API and web interface. Memory management is further optimized by using generators and iterators, which process data on-demand, rather than loading large datasets into memory at once.

Error Handling

In a project as intricate as "Resume Filter," robust error handling is crucial. The Python try-except blocks are systematically placed throughout the codebase to capture and handle exceptions, such as issues in file I/O, PDF parsing errors, or network-related failures during API calls. Logging mechanisms, powered by Python's logging module, are implemented to track critical errors, warnings, and debug information. The logs are designed to provide detailed traceback information, helping in the swift identification and resolution of bugs.

System Security Measures

Creation of User Profiles

The "Resume Filter" project adopts a modular approach to user profile creation by leveraging the 'allauth' authentication system in Django. User profiles are securely stored in the database, with critical information such as hashed passwords handled through Django's **make_password()** and **check_password()** utilities. Upon successful registration, user-specific data is assigned using **ForeignKey** relationships between the **User** model and other entities such as resumes processed and job applications. Upon deletion of User entire data associated with it will be deleted.

Managing the User Rights

User rights are managed through Django's built-in permissions and groups functionality, allowing for fine-grained control over who can perform specific actions. The project defines custom permissions for various models (e.g., resume processing, entity extraction) to ensure that users with differing roles have access only to the functionalities relevant to them. Administrators, for instance, have full control over the system, while recruiters may only have access to resume filtering and report generation features. Middleware functions monitor and log user activities, while the **@login_required** and **@permission_required** decorators in Django are employed to restrict access to critical views and API endpoints, ensuring secure user interactions within the system.

Cost Estimation

The **cost estimation** for the "Resume Filter" project involves analyzing the expenses incurred at various stages of development, deployment, and maintenance. These costs include both one-time expenditures and recurring costs associated with running the system efficiently.

Development Costs

The primary expense in the development phase revolves around the integration of Python-based technologies such as:

- **PDFMiner** or **PyMuPDF** (for PDF text extraction)
- **OpenAI GPT API** (used for entity extraction via ChatGPT)
- **Django Framework** (for API and Web UI development)

The implementation requires skilled developers with expertise in Python and related libraries to write code that is optimized for performance, scalability, and error handling. The cost here will be tied to the development time, which depends on the complexity of automation and the need for customizations in entity extraction.

Infrastructure Costs

To support the deployment of the "Resume Filter" project, several cloud services and infrastructure components must be factored into the cost model:

- **Cloud hosting** (such as AWS, Azure, or Google Cloud Platform) for deploying the Django web application and API services.
- **Database costs**, primarily related to using secure databases like **PostgreSQL** or **MySQL** for storing user information, resumes, and extracted entities.
- **Server maintenance** costs, including automatic scaling to handle increased volumes of resumes.

Cloud providers typically offer pricing models that charge based on **compute instances**, **data storage**, and **API call volume** (e.g., OpenAI API for ChatGPT). These recurring costs will vary depending on system usage, particularly in environments with high resume submission volumes.

API Usage Costs

A significant factor in the cost model is related to the **OpenAI GPT API** usage for entity extraction. Each API call generates cost based on token consumption, meaning the more resumes processed and the more extensive the extractions, the higher the usage costs. This must be considered carefully, especially when handling large-scale recruitment efforts.

Security Costs

Implementing system security features incurs additional costs, such as:

- **Encryption** for securing data both in transit and at rest using **SSL/TLS** certificates and secure database storage.
- **User Authentication and Authorization** using the **Django-allauth** package to ensure secure user login and rights management.

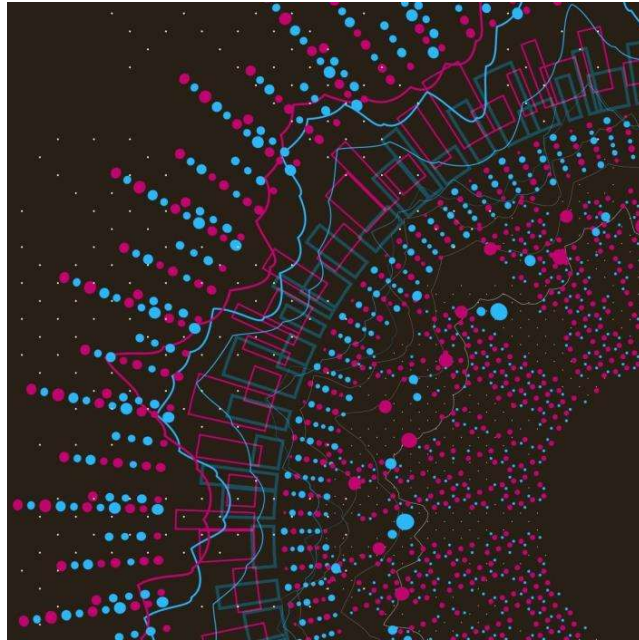
Appendices

The code and video of **resume filter** project can be found in the below github repository. Please go through the README.md file for setting up and running on your local machine.

Resume Filter: <https://github.com/madhubabukenchha/resume-filter>

Resume Filter

By
Kenchu Madhu Babu
Roll Num: 2214500471



Q1. What prompted you to choose this topic?



There is need for a system that could extract relevant information from resumes in a structured format, making it easier to compare and evaluate candidates.



I were interested in developing a tool that could automate the resume screening process, making it more efficient and timesaving for recruiters and employers.



I wanted to explore the potential of using ChatGPT for entity extraction in a practical application.



I believe that this project could have a positive impact on the recruitment process by making it more efficient and effective.

Q2. How the objectives specified in your study connected with the review of literature?

Objective	Connection to Literature Review
Use ChatGPT to extract entities from resumes	ChatGPT is an LLM that can be used for advanced NLP tasks, which is a promising approach for resume parsing.
Address the shortcomings of traditional resume parsing tools	The literature review identifies the limitations of traditional tools, such as their reliance on keyword matching and rule-based parsing.
Deliver a tangible and actionable output	The literature review discusses the need for resume parsing tools that are more user-friendly and provide more comprehensive information about candidates.

Q3. State the challenges faced by you while conducting the study for your seminar topic.

-  Limited access to real -world resume data: The study relied on publicly available datasets, which may not be representative of real -world resumes.
-  Fine-tuning ChatGPT for optimal performance: Fine-tuning a large language model like ChatGPT requires expertise and computational resources, which can be challenging.
-  Evaluating the effectiveness of the project: Evaluating the effectiveness of the project required comparing it to existing resume parsing tools, which can be difficult due to the lack of standardized benchmarks.
-  Ethical considerations of using ChatGPT for resume parsing: There are ethical considerations involved in using AI for resume screening, such as the potential for bias and discrimination.

Q4. Highlight the findings of your study conducted for this seminar.

- ChatGPT can be used to extract a wider range of entities from resumes compared to traditional tools.
- ChatGPT's contextual understanding leads to improved accuracy in entity extraction.
- "Resume Filter" has the potential to streamline the resume processing and analysis workflow for recruiters and HR professionals.



Q5. How do your findings bridge the gap between your seminar objectives and current industry practice?

- + Resume Filter bridges the gap between current industry practice and the seminar objectives by using advanced NLP techniques to extract a wider range of entities from resumes with greater accuracy. This approach addresses the limitations of traditional resume parsing tools, which are often inaccurate and limited in the scope of information they extract.

Q6. State the conclusion and future scope of the seminar topic.



Fine-tuning ChatGPT for specific domains or industries.



Integrating sentiment analysis to gauge candidate enthusiasm.



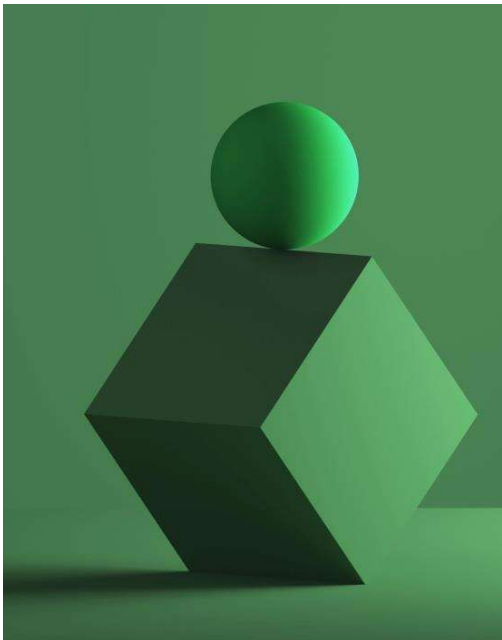
Developing an interactive interface for real-time entity exploration.



Skills, achievements, soft skills, and more, not just basic details.



Seamless integration for wider adoption and impact



Thank you